

6. Regression Analysis

*“Predicting the future isn't magic,
it's artificial intelligence.”
— Dave Waters*

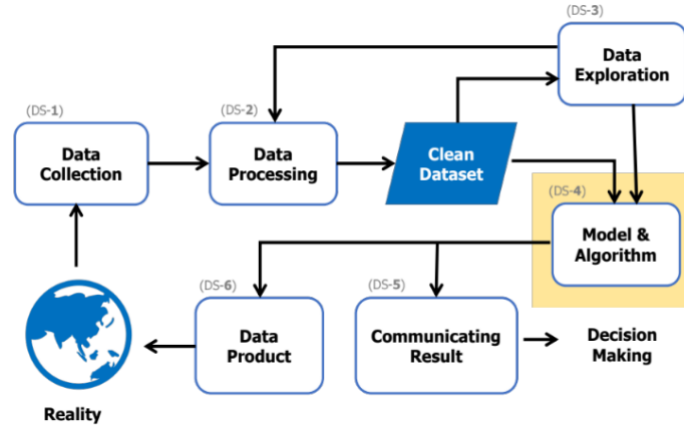


Figure 6.0-1: Regression analysis in the data science process

Regression is a predictive-analytics technique within supervised learning, one approach to the Model Development (DS-4) stage (Figure 6.0-1). It predicts interval- or ratio-scale continuous values, or ordinal data once encoded as numbers. Regression is widely used in everyday life — forecasting temperature, stock prices, speed, and more.

This chapter covers linear regression, evaluating regression results, feature selection, data scaling, and the process for developing a linear regression model.

6.1 Linear Regression

Linear regression takes the familiar high-school form:

$$y = mx + c$$

where y is the output, m is the line's slope (a constant), x is the input, and c is the constant where the line crosses the y -axis.

In data analytics, this is instead written as:

$$f(x) = b_0 + b_1x$$

or equivalently $\hat{y} = b_0 + b_1x$, where $f(x)$ or $\hat{y}$ is the output, x is the input, b_0 is the intercept (where the line crosses the y -axis), and b_1 is the coefficient of x .

For example, a model predicting a monthly phone bill might be:

$$\text{phone bill} = 100 + 2 \times \text{call minutes}$$

So 20 minutes of calls gives a bill of $100 + 2 \times 20 = 140$ baht.

With multiple input variables, the equation extends to:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

where X (capital) denotes the full set of inputs $x_1, x_2, x_3, \dots, x_n$. Extending the phone-bill example with texts and data usage:

$$\text{phone bill} = 100 + 2 \times \text{call minutes} + 1 \times \text{texts sent} + 50 \times \text{data usage (GB)}$$

So 20 minutes of calls, 5 texts, and 3GB of data gives $100 + 2 \times 20 + 1 \times 5 + 50 \times 3 = 295$ baht.

In data analytics, though, a linear regression model must be built by learning from data, as explained next.

Learning to Build a Linear Regression Model

Suppose we have height data for 4 students (Table 6.1-1), with columns for student, weight (kg), and height (cm). This example builds a linear regression model to predict height from weight.

Table 6.1-1: Example weight and height data for 4 students

student	weight	height
s1	50	170
s2	55	165
s3	60	180
s4	65	175

The analyst must build a model from the equation $\hat{y} = b_0 + b_1x_1$, where $\hat{y}$ is predicted height, x_1 is weight (input), b_0 is the intercept, and b_1 is the coefficient of x_1 . The analyst's job is to find the b_0 and b_1 that let the model predict height from weight as accurately as possible.

Plotting weight (x-axis) against height (y-axis) from Table 6.1-1 gives the scatter plot in Figure 6.1-1.

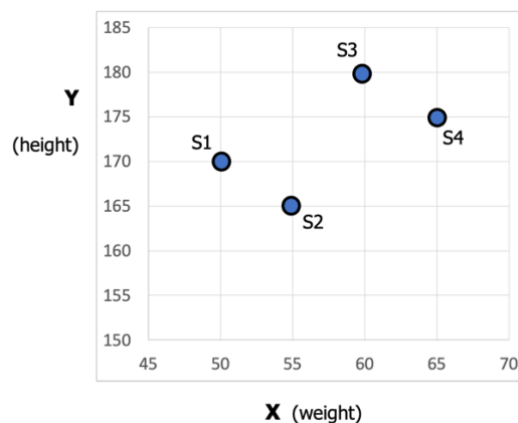


Figure 6.1-1: Scatter plot of student weight and height

Many different linear regression lines could fit this data, as shown in Figure 6.1-2:

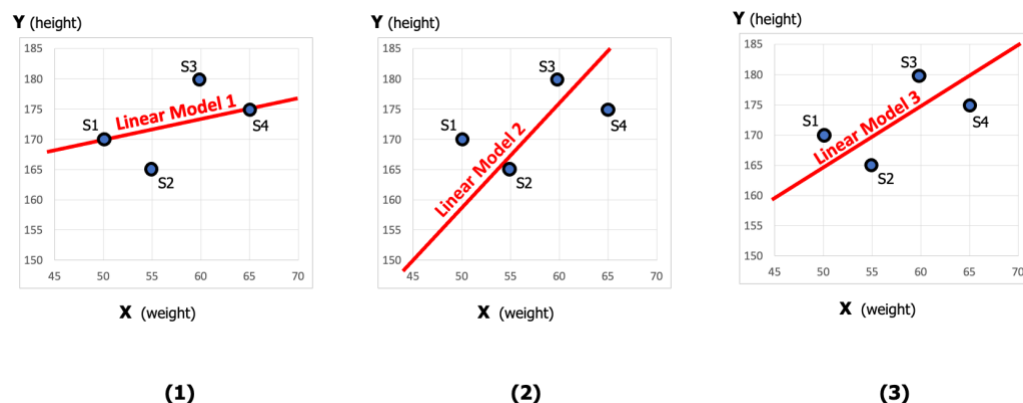


Figure 6.1-2: Example candidate linear regression models for predicting student height

- Model 1: $\hat{y} = 167 + 0.33x_1$
- Model 2: $\hat{y} = 150 + 1.75x_1$
- Model 3: $\hat{y} = 160 + 1.00x_1$

With multiple candidates, which is best? The one whose predictions come closest to the true values — i.e., the smallest error. Evaluation methods are covered in section 6.2.

Learning a linear regression model from data typically uses the Gradient Descent algorithm, gradually adjusting each coefficient (b_1 , etc.) and the intercept (b_0).

Starting from randomly chosen b_0 and b_1 , the algorithm makes small adjustments and checks whether the error increases or decreases. If an adjustment reduces error, the next round continues adjusting in the same direction; if it increases error, the next round adjusts in the opposite direction — repeating over many rounds until the error is minimized or changes only negligibly (Bishop & Nasrabadi, 2006).

Rather than adjusting by a fixed amount each round, the adjustment size depends on the ratio of error change to adjustment size — in calculus terms, the first derivative of the loss function (related to a continuous evaluation measure such as MSE), multiplied by a constant called the learning rate. If the learning rate is too large, each step is large — like taking big strides toward a destination: fast, but liable to overshoot. If too small, each step is tiny — slow, but less likely to overshoot. Choosing this value well matters, and it's itself part of the experimental environment for many techniques.

6.2 Evaluating Regression Analysis

Regression output is a continuous numeric prediction, so evaluation centers on how different the predicted value is from the actual value — the error. A perfectly accurate model has zero error; a highly inaccurate model has large error.

Regression analysis therefore aims to keep error as close to zero as possible — in short, smaller error is better.

This book covers three popular error measures: MAE (Mean Absolute Error), MAPE (Mean Absolute Percentage Error), and RMSE (Root Mean Square Error).

Mean Absolute Error (MAE)

MAE is the simplest error measure, computed as follows.

Steps

1. Compute the error for each prediction: actual value (y_{test}) minus predicted value (y_{pred}).
2. Take the absolute value of each error.
3. Average the absolute errors.

Equation

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where i is the i -th example; y_i is the actual test value (y_{test} in code); $\hat{y}_i$ is the predicted value (y_{pred} in code); $y_i - \hat{y}_i$ is the error; the absolute value ensures a positive result; and n is the total number of examples.

Worked Example

Using Table 6.2-1, with 3 examples (predicted via $y = 3 + 2x_1 + 4x_2 + 2x_3$):

Table 6.2-1: Example predictions for evaluation

#	x1	x2	x3	y (actual)	$\hat{y}$ (predicted)
1	1	1	1	9	11
2	2	1	1	12	13
3	1	1	3	17	15

$$\begin{aligned}
 MAE &= \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \\
 &= \frac{1}{3} (|y_1 - \hat{y}_1| + |y_2 - \hat{y}_2| + |y_3 - \hat{y}_3|) \\
 &= \frac{1}{3} (|9 - 11| + |12 - 13| + |17 - 15|) \\
 &= \frac{1}{3} (|-2| + |-1| + |2|) \\
 &= \frac{1}{3} (2 + 1 + 2) \\
 &= \frac{1}{3} (5) \\
 &= 1.67
 \end{aligned}$$

Code Example

```
import pandas as pd

y_test = pd.Series([9, 12, 17])
y_pred = pd.Series([11, 13, 15])
mae = (y_pred - y_test).abs().mean()
print("MAE =", mae)

MAE = 1.666666666666667
```

Scikit-Learn also provides `mean_absolute_error` directly:

```
import pandas as pd
from sklearn.metrics import mean_absolute_error

y_test = pd.Series([9, 12, 17])
y_pred = pd.Series([11, 13, 15])
mae = mean_absolute_error(y_test, y_pred)
print("MAE =", mae)

MAE = 1.666666666666667
```

Mean Absolute Percentage Error (MAPE)

MAPE reports error as a percentage, similar to MAE but with an added percentage-difference step.

Steps

1. Compute the error for each prediction: `y_test` minus `y_pred`.
2. Compute the percentage change relative to the actual value.
3. Take the absolute value.
4. Average the results.

Equation

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left(\left| \frac{y_i - \hat{y}_i}{y_i} \right| \right) \times 100$$

MAPE adds a division by y_i to get the proportional change, then multiplies by 100 for a percentage.

Worked Example

$$\begin{aligned} MAPE &= \frac{1}{n} \sum_{i=1}^n \left(\left| \frac{y_i - \hat{y}_i}{y_i} \right| \right) \times 100 \\ &= \frac{1}{3} \left(\left| \frac{y_1 - \hat{y}_1}{y_1} \right| + \left| \frac{y_2 - \hat{y}_2}{y_2} \right| + \left| \frac{y_3 - \hat{y}_3}{y_3} \right| \right) \times 100 \\ &= \frac{1}{3} \left(\left| \frac{9 - 11}{9} \right| + \left| \frac{12 - 13}{12} \right| + \left| \frac{17 - 15}{17} \right| \right) \times 100 \\ &= \frac{1}{3} (|-0.22| + |-0.08| + |0.12|) \times 100 \\ &= \frac{1}{3} (0.22 + 0.08 + 0.12) \times 100 \\ &= \frac{1}{3} (0.42) \times 100 \\ &= 14 \% \end{aligned}$$

Code Example

Scikit-Learn has no built-in MAPE function, so it's computed manually:

```
import pandas as pd

y_test = pd.Series([9, 12, 17])
y_pred = pd.Series([11, 13, 15])
mape = ((y_pred - y_test) / y_test).abs().mean() * 100
print("MAPE =", mape, "%")

MAPE = 14.106753812636164 %
```

Root Mean Square Error (RMSE)

RMSE is a popular measure, especially in research, computed as follows.

Steps

1. Compute the error for each prediction: y_{test} minus y_{pred} .
2. Square each error.

3. Average the squared errors.
4. Take the square root.

Equation

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Similar to MAE, but squaring the error (instead of taking its absolute value), then averaging and taking the square root.

Note: without the square root, this is called MSE (Mean Squared Error).

Worked Example

$$\begin{aligned}
 RMSE &= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \\
 &= \sqrt{\frac{1}{3} ((y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2)} \\
 &= \sqrt{\frac{1}{3} ((9 - 11)^2 + (12 - 13)^2 + (17 - 15)^2)} \\
 &= \sqrt{\frac{1}{3} ((-2)^2 + (-1)^2 + (2)^2)} \\
 &= \sqrt{\frac{1}{3} (4 + 1 + 4)} \\
 &= \sqrt{\frac{1}{3} (9)} \\
 &= 1.73
 \end{aligned}$$

Code Example

```
import pandas as pd
import math

y_test = pd.Series([9, 12, 17])
y_pred = pd.Series([11, 13, 15])
rmse = math.sqrt(((y_pred - y_test)**2).mean())
print("RMSE =", rmse)

RMSE = 1.7320508075688772
```

Scikit-Learn has no direct RMSE function, but `mean_squared_error` with `squared=False` gives the same result:

```
import pandas as pd
from sklearn.metrics import mean_squared_error

y_test = pd.Series([9, 12, 17])
y_pred = pd.Series([11, 13, 15])
rmse = mean_squared_error(y_test, y_pred, squared=False)
print("RMSE =", rmse)

RMSE = 1.7320508075688772
```

6.3 Selecting Input Variables (Features)

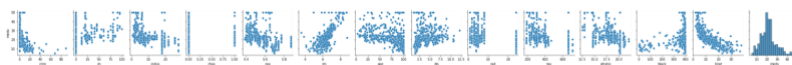
Model building draws on multiple input variables, called features. Using several features often improves accuracy over just one, but not every feature is suitable. Feature selection for linear relevance to the target typically combines two approaches: examining scatter plots and computing correlation.

Scatter Plot

Seaborn is recommended for viewing multiple variables against a target at once:

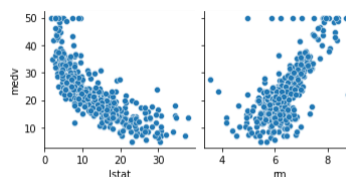
```
import pandas as pd
import seaborn as sns

df = pd.read_csv('https://rathachai.github.io/DA-LAB/datasets/boston.csv')
sns.pairplot(df, x_vars=boston.columns, y_vars=["medv"])
```



Reviewing all features against the target `medv`, two — `lstat` and `rm` — show a fairly linear relationship, so they're plotted again for confirmation:

```
sns.pairplot(df, x_vars=["lstat", "rm"], y_vars=["medv"])
```



Correlation Analysis

To sharpen feature selection, compute Pearson correlation between every feature and the target — easily done via Pandas:

```
df.corr().sort_values("medv")["medv"]
```

```
lstat      -0.737663
ptratio    -0.507787
indus      -0.483725
tax        -0.468536
nox        -0.427321
crim       -0.388305
rad        -0.381626
age        -0.376955
chas       0.175260
dis        0.249929
```

black	0.333461
zn	0.360445
rm	0.695360
medv	1.000000

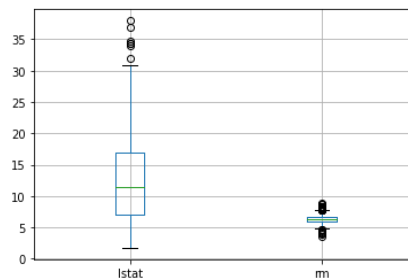
Feature selection favors correlation values close to 1 or -1 — close to 1 means a strong positive linear relationship, close to -1 a strong negative one; either can be used. Here, lstat and rm are good candidates (medv itself is excluded, being the target).

6.4 Scaling Data

After selecting features, check their value ranges for scaling. If features share similar ranges, they can be used directly; if ranges differ greatly — e.g., one in the tens, another in the tens of thousands — this can distort linear regression's learning process.

Checking ranges can use Pandas' box plot, continuing the previous example:

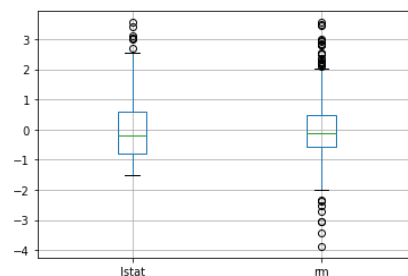
```
df[["lstat", "rm"]].boxplot()
```



The two features clearly differ in range. Next, scale them to a common range using Scikit-Learn's Standard Scaler:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(df[["lstat", "rm"]])
X_scaled = scaler.transform(df[["lstat", "rm"]])
pd.DataFrame(X_scaled, columns=["lstat", "rm"]).boxplot()
```



After scaling, both features fall in a similar range, ready for the linear regression technique to learn from.

In practice, data is split into a Training Set and Test Set (X_{train} , X_{test}). Scaling then follows these key steps:

5. Build the scaler by fitting it only on X_{train} — never on the full dataset, and treat X_{test} as unseen at this stage.
6. Scale X_{train} using the fitted scaler.
7. Scale X_{test} using the same scaler (fitted on X_{train}), then use the scaled X_{test} for prediction.

Scaling normally applies only to features, not the target (y), since scaling y adds little benefit and risks errors when converting predictions back to the original scale.

6.5 Steps in Developing a Linear Regression Model

Since linear regression is simple, it's commonly used to illustrate the overall model-building process, from importing data through testing the model. This section uses the Boston housing dataset (section 2.5.4), with Python code throughout.

Import Data

```
import pandas as pd

df = pd.read_csv('https://rathachai.github.io/DA-LAB/datasets/boston.csv')
```

Select Features

Show Pearson correlation of every feature against the target medv, sorted ascending:

```
df.corr().sort_values("medv")["medv"]
```

(same output as above, omitted for brevity)

Assign X to the selected features (lstat, rm), since their correlations are close to 1 and -1, and y to the target (medv, median home value in \$1,000s):

```
X = df[["lstat", "rm"]]
y = df["medv"]
```

Build and Test the Model

Import the necessary functions, mostly from Scikit-Learn:

```
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

Then build and test via K-Fold Cross-Validation:

```
kf = KFold(n_splits=5)
round_num = 1
RMSEs = []
for train_index, test_index in kf.split(X):
    X_train, X_test = X.loc[train_index], X.loc[test_index]
    y_train, y_test = y.loc[train_index], y.loc[test_index]
    scaler = StandardScaler().fit(X_train)
    X_train_scaled = scaler.transform(X_train)
    model = LinearRegression()
    model.fit(X_train_scaled, y_train)
    X_test_scaled = scaler.transform(X_test)
    y_pred = model.predict(X_test_scaled)
    rmse = mean_squared_error(y_test, y_pred, squared=False)
    print("Round", round_num, " RMSE = ", rmse)
    round_num += 1
    RMSEs.append(rmse)

Round 1 RMSE = 3.5737650221240846
Round 2 RMSE = 4.3961400739566745
Round 3 RMSE = 6.628536351157885
Round 4 RMSE = 7.48609952359262
Round 5 RMSE = 5.097658364269636
```

Summarize Overall Results

```
import numpy as np

rmse_mean = np.array(RMSEs).mean()
print("Mean of RMSE = ", rmse_mean)

Mean of RMSE = 5.436439867020181
```

Build the Final Model on the Full Dataset

Satisfied with this environment — linear regression, default parameters, and features lstat and rm — build the production model on the full dataset:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

df = pd.read_csv('https://rathachai.github.io/DA-LAB/datasets/boston.csv')
X = df[["lstat", "rm"]]
y = df["medv"]
scaler = preprocessing.StandardScaler().fit(X)
X_scaled = scaler.transform(X)
model = LinearRegression()
model.fit(X_scaled, y)
print("Features :", X.columns.tolist())
print("Coefficients:", model.coef_)
print("Intercept :", model.intercept_)

Features : ['lstat', 'rm']
Coefficients: [-5.81415734  2.52530742]
Intercept : 22.532806324110677
```

Save the scaler and model for deployment. The resulting model is:

$$medv = 22.53 - 5.81 \cdot lstat + 2.53 \cdot rm$$

So, given lstat and rm, medv can be predicted, with model performance measured at roughly RMSE 5.44 via 5-Fold Cross-Validation.

6.6 The Technical Mechanism

Behind machine learning — including linear regression — lies Gradient Descent, explained below.

Data Representation

Tabular data can be viewed as matrices and vectors: X is the input matrix, with element $x_j^{(i)}$ for row i (example) and column j (attribute) — the first column, $x_0^{(i)}$, is fixed at 1 for all rows. y is the output vector, with element $y^{(i)}$ for row i :

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ 1 & x_1^{(3)} & x_2^{(3)} \end{bmatrix}, \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \end{bmatrix}$$

Linear Regression Equation

The prediction for row i , $\hat{y}^{(i)}$, with θ_j as the coefficient for x_j , needing to be optimized so predictions differ as little as possible from actual values:

$$\widehat{y^{(i)}} = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)}$$

Viewed as vectors, $\hat{y}$ and θ give the compact form:

$$\hat{y} = \theta^T X$$

or

$$\begin{bmatrix} \widehat{y^{(1)}} \\ \widehat{y^{(2)}} \\ \widehat{y^{(3)}} \end{bmatrix} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}^T \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ 1 & x_1^{(3)} & x_2^{(3)} \end{bmatrix}$$

Defining the Model's Objective

Mean Squared Error is commonly used as the loss function, since it's a smooth, continuous (parabolic) function suited to derivative-based minimization. Here, m is the total number of examples, and dividing by 2 simplifies later derivative calculations:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Updating Linear Regression Coefficients

Coefficients are refined via Gradient Descent, where the θ_j on the left is the newly updated coefficient, the θ_j on the right is its previous value, and the partial-derivative term is the adjustment, scaled by the learning rate α :

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Applying the chain rule to the derivative term gives, after working through the algebra:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \left(\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 \right) \\ &= \frac{1}{2m} \sum_{i=1}^m \left(\frac{\partial}{\partial \theta_j} (\hat{y}^{(i)} - y^{(i)})^2 \right) \\ &= \frac{1}{2m} \sum_{i=1}^m \left(\frac{\partial}{\partial \hat{y}^{(i)}} (\hat{y}^{(i)} - y^{(i)})^2 \cdot \frac{\partial \hat{y}^{(i)}}{\partial \theta_j} \right) \end{aligned}$$

The $\frac{1}{2} \frac{\partial}{\partial \hat{y}^{(i)}} (\hat{y}^{(i)} - y^{(i)})^2$ can be calculated by:

$$\begin{aligned} \frac{1}{2} \frac{\partial}{\partial \hat{y}^{(i)}} (\hat{y}^{(i)} - y^{(i)})^2 &= \frac{1}{2} 2 (\hat{y}^{(i)} - y^{(i)}) \cdot 1 \\ &= (\hat{y}^{(i)} - y^{(i)}) \end{aligned}$$

Then, calculate $\frac{\partial}{\partial \theta_j} \hat{y}^{(i)}$ from $\hat{y}^{(i)} = \theta_0 x_0^{(i)} + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)}$, so the equation can be:

$$\frac{\partial}{\partial \theta_j} \hat{y}^{(i)} = \frac{\partial}{\partial \theta_j} (\theta_0 x_0^{(i)} + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)})$$

Considering θ_1 that other θ are constants, then $\frac{\partial}{\partial \theta_1} \widehat{y^{(i)}} = x_1^{(i)}$ can be:

$$\frac{\partial}{\partial \theta_0} \widehat{y^{(i)}} = x_0^{(i)} = 1, \quad \frac{\partial}{\partial \theta_1} \widehat{y^{(i)}} = x_1^{(i)} \quad \text{and} \quad \frac{\partial}{\partial \theta_2} \widehat{y^{(i)}} = x_2^{(i)}$$

Then, the derivative equation can be:

$$\theta_j = \theta_j - \alpha \cdot \frac{1}{m} \sum_{i=1}^m ((\widehat{y^{(i)}} - y^{(i)}) \cdot x_j^{(i)})$$

And the equation after adapting the coefficients can be:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m ((\widehat{y^{(i)}} - y^{(i)}) \cdot x_j^{(i)})$$

Or, in the term of matrix where x_j is the vector of independent variable at the column j

$$\theta_j = \theta_j - \alpha \cdot \frac{1}{m} (\widehat{y^{(i)}} - y^{(i)})^T x_j$$

Worked Example

Suppose we have the data below, with two inputs x_1, x_2 and output y :

Table 6.6-1: Example data for linear regression calculation

x_1	x_2	y
1	2	13
2	3	19
3	4	25
4	5	31

Adding a first column x_0 fixed at 1:

$$X = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 4 \\ 1 & 4 & 5 \end{bmatrix}, \quad y = \begin{bmatrix} 13 \\ 19 \\ 25 \\ 31 \end{bmatrix}$$

Randomly initializing $\theta_0=2, \theta_1=3, \theta_2=1$ gives predictions:

$$\hat{y} = \theta^T X = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}^T \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 4 \\ 1 & 4 & 5 \end{bmatrix} = \begin{bmatrix} 7 \\ 11 \\ 15 \\ 19 \end{bmatrix}$$

$$\theta_j^{next} = \theta_j - \alpha \cdot \frac{1}{m} (\widehat{y^{(i)}} - y^{(i)})^T x_j$$

Choosing a small learning rate, e.g., $\alpha=0.01$, and applying the update rule (with $m=4$ examples):

$$\theta_0^{next} = 2 - 0.01 \times \frac{1}{4} \left(\begin{bmatrix} 7 \\ 11 \\ 15 \\ 19 \end{bmatrix} - \begin{bmatrix} 13 \\ 19 \\ 25 \\ 31 \end{bmatrix} \right)^T \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = 2.09$$

$$\theta_1^{next} = 3 - 0.01 \times \frac{1}{4} \left(\begin{bmatrix} 7 \\ 11 \\ 15 \\ 19 \end{bmatrix} - \begin{bmatrix} 13 \\ 19 \\ 25 \\ 31 \end{bmatrix} \right)^T \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} = 3.25$$

$$\theta_2^{next} = 1 - 0.01 \times \frac{1}{4} \left(\begin{bmatrix} 7 \\ 11 \\ 15 \\ 19 \end{bmatrix} - \begin{bmatrix} 13 \\ 19 \\ 25 \\ 31 \end{bmatrix} \right)^T \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} = 1.34$$

This process runs in rounds (epochs). After round 1, the model is $\hat{y}^{(i)} = 2.09 + 3.25 \cdot x_1^{(i)} + 1.34 \cdot x_2^{(i)}$, with RMSE 9.27. The coefficients are then updated again, repeating until a stopping condition is met — a fixed number of rounds, or error falling below a threshold. The Python code below runs 1,000 rounds, using b_0 , b_1 , b_2 for the coefficients:

```
import numpy as np

x0 = np.array([1,1,1,1])
x1 = np.array([1,2,3,4])
x2 = np.array([2,3,4,5])
y = np.array([13,19,25,31])
alpha = 0.01
b0, b1, b2 = 2, 3, 1
for i_epoch in range(1000):
    y_pred = b0*x0 + b1*x1 + b2*x2
    b0 = b0 - alpha*((y_pred-y)*x0).mean()
    b1 = b1 - alpha*((y_pred-y)*x1).mean()
    b2 = b2 - alpha*((y_pred-y)*x2).mean()
```

After 1,000 rounds, the model becomes $\hat{y}^{(i)} = 3.713 + 3.207 \cdot x_1^{(i)} + 2.929 \cdot x_2^{(i)}$, with RMSE 0.15. Plotting each round's coefficient values (Figure 6.6-1) shows error steadily decreasing as coefficients are refined, converging to an increasingly accurate model.

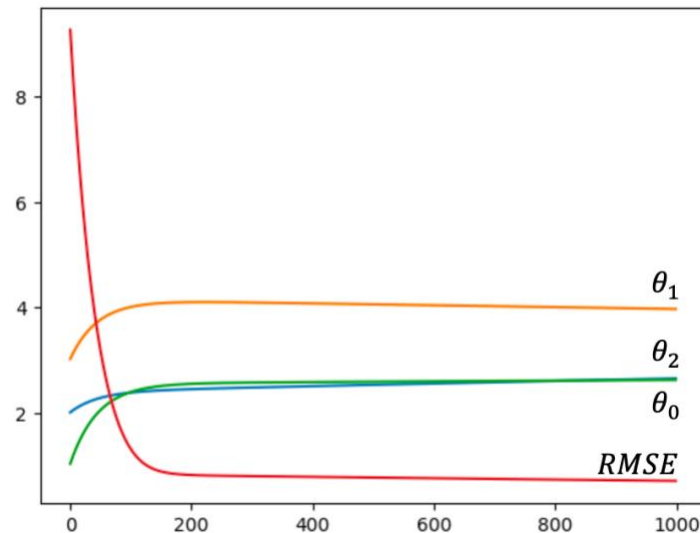


Figure 6.6-1: Coefficient adjustment over successive rounds

6.7 Chapter Summary

This chapter covered regression analysis using linear regression, for predicting interval- or ratio-scale continuous values (or encoded ordinal data). It began with the linear regression equation and learning a model from data, then evaluation methods — MAE, MAPE, and RMSE — followed by the model-building process: feature selection via correlation analysis, scaling inputs to a common range, and finally the full model-building process with Python examples, illustrating how a linear regression model is built end to end.

6.8 Review Questions

1. Explain the components of the following linear regression model:

$$\hat{y} = 10 - 3x_1 + 5x_2 + 7x_3$$

2. If a built model has an RMSE close to 1, discuss whether it's accurate enough for production use, with examples.
3. Using the model $\hat{y} = 160 + 1 \cdot \text{weight}$ to predict height from Table 6.1-1 (student weight/height data), show the calculations for MAE, MAPE, and RMSE.
4. Using the data in Table 6.8-1, compute the Pearson correlation of each input variable against y .
5. Using the data in Table 6.8-1, write Python code to build a linear regression model predicting y from the inputs, show the production-ready model, and report RMSE via 4-Fold Cross-Validation.
6. Using the data in Table 6.8-1 and Gradient Descent, assuming all coefficients start at 1: (a) write X and y as a matrix and vector; (b) predict y and compute RMSE; (c) show the first coefficient update, then predict y and compute RMSE; (d) show the second coefficient update, then predict y and compute RMSE.
7. Using the data in Table 6.8-1, write Python code to compute the coefficients using only the NumPy library, and plot a line chart of the coefficient updates.

Table 6.8-1: Data for the linear regression review questions

x1	x2	x3	x4	y
25	62	229	47	338
19	90	200	29	377
10	149	127	17	262
31	77	120	46	144
30	157	271	13	363
41	130	144	34	130
33	167	124	22	137
10	61	101	25	196

6.9 References

Bishop, C. M., & Nasrabadi, N. M. (2006). Pattern Recognition and Machine Learning (Vol. 4, No. 4). Springer.

Boschetti, A., & Massaron, L. (2015). Python Data Science Essentials. Packt Publishing Ltd.

Brunton, S. L., & Kutz, J. N. (2022). Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control. Cambridge University Press.

Igual, L., & Seguí, S. (2017). Introduction to Data Science. Springer, Cham.

Ng, A. (2023). Supervised Machine Learning: Regression and Classification [MOOC]. Coursera. <https://www.coursera.org/learn/machine-learning>

O’Neil, C., & Schutt, R. (2013). Doing Data Science: Straight Talk from the Frontline. O’Reilly Media, Inc.

Provost, F., & Fawcett, T. (2013). Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking. O’Reilly Media, Inc.

Russell, S., & Norvig, P. (2002). Artificial Intelligence: A Modern Approach.

VanderPlas, J. (2016). Python Data Science Handbook: Essential Tools for Working with Data. O’Reilly Media, Inc.

Zhang, T. (2023). Mathematical Analysis of Machine Learning Algorithms. Cambridge University Press.